

Pothi: owning your bibliography

*Your reference manager belongs in your folder,
not in someone else's cloud.*

Kanchan Sarkar

pcksarkar@gmail.com

May 10, 2026

Abstract

Most reference managers — Zotero, Mendeley, EndNote — treat your library as one big pile and lock it inside their software. **Pothi** is built differently: each paper you are *writing* is its own folder on disk, and that folder owns its own auto-updating `references.bib`. Drop a PDF into the folder and Pothi finds it, looks up the metadata from CrossRef, and adds the entry to your library. **If you write in LaTeX**, Pothi keeps `references.bib` in your manuscript's folder current as you cite — your next `pdflatex` run picks up the change with no manual export step. **If you write in Word**, type `[@SmithCrystal2024]` in your document, drop the docx into Pothi, and out comes a polished version with formatted citations and a bibliography. No account to make, no server to run, no subscription — your library is plain JSON sitting in a folder you chose. This document explains the design choices behind the tool, walks through five things you would actually do with it (add a paper by DOI, drag in a PDF, build a manuscript bibliography for LaTeX or Word, recover after switching browsers), compares it honestly to the popular options, and includes the installation and usage manual. The code is open under the MIT license. I wrote it for myself because the existing tools did not fit how I work — sharing it because it might fit you too.

1 Why I built this

I write papers in LaTeX, sometimes Word. My references live in PDFs scattered across several folders on my laptop and my Nextcloud. I tried Zotero and Mendeley over the years; both eventually frustrated me for the same reasons.

- **Cloud lock-in.** Both want my data in their cloud. Mendeley sold to Elsevier; Zotero is community-run but its sync is server-side. I have lost a Mendeley library before to a sync conflict.
- **Library-first model.** Both organize the world as one global library that I then “use” to cite from. But when I am writing a paper, the unit of work is *the paper*, not the library. I want this paper's bibliography to be a curated, ordered list with my own notes about why each reference is in — and I want that list to be a plain file next to my manuscript, not a tag inside a database.
- **Attachment fragility.** Rename a PDF on disk and the link in Zotero breaks. Move a folder and dozens of links go stale.

- **Vendor pace.** EndNote costs money. Papers and ReadCube changed hands. JabRef is great if you write in LaTeX; awkward otherwise.
- **The .bib file should live next to my manuscript.** When I write a LaTeX paper, the natural unit is the folder I `pdflatex` from. I want `references.bib` to sit in that folder, kept current as I cite, and to be the same file I email a co-author. Most tools instead want a global library that I “cite from” through a plugin.

I am a researcher, not a software person. So this is not a polished commercial tool; it is a thing I wrote for myself in a few hundred hours over a few weeks. It runs entirely in the browser, has no backend, and stores everything in a folder you pick. If it fits how you work, take it. If not, the existing tools are right there.

2 Design principles

Five principles kept the project small and consistent.

2.1 Local-first

The data is yours. There is no account, no server, no subscription. The library is plain JSON in a folder you control. If Pothi disappears, your files are still there. Inspired by the Local-First Software essay (Kleppmann et al., Ink & Switch, 2019).

2.2 Manuscript-first

A reference is interesting because of what *paper* you cite it from. Pothi treats each manuscript as a first-class object: it has a name, an associated folder, an ordered list of cited references, and a per-citation rationale (“why is this cited?”). The global library is the union of all manuscript bibliographies, not the other way around.

2.3 Plain text on disk

When you link a folder, Pothi auto-writes `_pothi-library.json` (the whole library) and per-manuscript `references.bib` / `refs.json` files into it. Open them in any text editor. Track them in git. Email them. The browser’s IndexedDB is a cache, not the source of truth.

2.4 Schema as config

Entry types, required fields, citekey templates — they all live in `js/schema.js` as a plain configuration object. To add a custom entry type or field, edit the file. There is no plugin API to learn.

2.5 No build step

The project is HTML, CSS, and ES module JavaScript. Edit a file, refresh the browser. Vendored libraries (Preact, htm, PDF.js, fflate) live in `vendor/` as committed modules. There is no `npm install`, no webpack, no transpilation.

3 How it is put together

The full project is one folder of static files.

```
Pothi/
index.html # shell + import map
css/
  variables.css # design tokens
  base.css, layout.css, components.css
js/
  schema.js # entry types + fields + citekey template
  db.js # IndexedDB wrapper (entries, manuscripts, meta)
  bibtex.js # parser + emitter, with LaTeX -> Unicode
  citekey.js # {Author1}{year}{title3} template engine
  lookup.js # CrossRef + arXiv (via DOI) + OpenLibrary + Sem.Sch.
  folder.js # File System Access API wrapper
  folder-scan.js # recursive walk + SHA-256 dedup
  pdf-extract.js # PDF.js + DOI/arXiv regex
  pdf-preview.js # in-app PDF viewer
  docx.js # Pandoc-bridge equivalent (fflate + DOMParser)
  styles.js # author-year + numeric citation styles
  library-backup.js # export/import + auto-write to folder
  smart-collections.js # saved searches
  manuscripts.js # manuscript helpers
  markdown.js # tiny safe Markdown -> DOM
  tag-suggest.js # TF-IDF tag suggestions
  app.js # main render
vendor/ # Preact, htm, PDF.js, fflate (vendored)
install.sh, install.bat # launcher installers
LICENSE # MIT
```

The browser entry point is one HTML file with an *import map* that points each module's name at its vendored file:

```
<script type="importmap">
{ "imports": {
  "preact": "./vendor/preact.module.js",
  "preact/hooks": "./vendor/preact-hooks.module.js",
  "htm": "./vendor/htm.module.js"
} }
</script>
```

IndexedDB stores three object stores: **entries**, **manuscripts**, and a **meta** key/value singleton (where the linked-folder `FileSystemDirectoryHandle` lives across reloads).

The browser refuses ES modules from `file://` for security, so Pothi needs an HTTP server. It ships an installer (`install.sh` on Linux/ macOS, `install.bat` on Windows) that drops a `pothi` command on your PATH: typing `pothi` in any terminal starts a Python `http.server` on `127.0.0.1:8765` and opens your browser there.

4 Five things you would actually do with it

4.1 Add a paper by DOI / arXiv ID / ISBN / URL

Click *+ New*. Paste any of: a bare DOI (10.1038/nature14539), a doi: prefix, a full URL (<https://doi.org/10.x/y> or any journal page that has a DOI in it — the input sniffs out the DOI), an arXiv ID (2401.04088), or an ISBN (9780198506058). Press Enter. The handler routes to CrossRef / arXiv-via-CrossRef / OpenLibrary as appropriate. Title, authors (in BibTeX *Last, First* form), journal, volume, issue, pages, year, DOI all populate. When CrossRef has no abstract, Pothi falls back to Semantic Scholar to fill it. A citekey is generated from {Author1}{year}{title3} (configurable). If a duplicate DOI is already in your library, you get a clear warning before adding.

4.2 Drag a PDF anywhere on the page

A blue overlay appears: “Drop PDFs or .docx here.” Release. Pothi:

1. extracts the embedded text and XMP metadata from the PDF (PDF.js);
2. regex-sniffs for a DOI on the first three pages;
3. if found, calls CrossRef to fetch full metadata;
4. generates a citekey;
5. if a library folder is linked, copies the PDF into it as <citekey>.pdf and attaches it to the new entry.

Hit rate in practice is around 85% for journal articles and higher for arXiv preprints. PDFs without a DOI get the title from the PDF’s embedded metadata; you fill the rest.

If the PDF’s content hash matches a file already attached to an entry, the import is silently de-duplicated.

4.3 Build a manuscript bibliography

Click *+ New manuscript* in the sidebar. Name it (*Polymorphism review 2026*). The main pane switches to a *manuscript view*: a search box at the top, the bibliography below.

Search the library inline; click *+ Cite* to add a reference. A textarea appears next to each cited entry where you write the rationale — “cited in §3.2 because Bernstein establishes the canonical polymorphism framework.” Reorder with up / down buttons. Switch the citation style between *Author-year* (APA-ish) and *Numeric* (IEEE-ish).

Click *+ Link folder* on the manuscript to point it at the folder that holds your draft (manuscripts can each have a different folder). Pothi then auto-writes `references.bib` and `refs.json` into that folder on every change, debounced 4 seconds.

If you write in LaTeX. This is the whole workflow:

```
% In your manuscript folder, after linking it to the manuscript:
\documentclass{article}
\usepackage{biblatex}
\addbibresource{references.bib} % Pothi keeps this current
```

```

\begin{document}
As Bernstein argues~\cite{Bernstein2020Polymorphism}, ...
% Or for vanilla BibTeX:
% \bibliography{references}
\printbibliography
\end{document}

```

Run `pdflatex` as usual. Each time you cite or uncite in Pohti, the `references.bib` next to your `.tex` updates within seconds; the next compile sees the change. No export step, no Cite-While-You-Write plugin, no app to keep running in the background. The `.bib` file is the contract; both Pohti and your TeX engine read and write it as plain text.

For BibLaTeX users, `references.bib` is BibLaTeX-clean (the emitter handles Unicode, doesn't lossy-escape, preserves brace protection). For BibTeX (8-bit) users, your usual `inputenc` setup works.

4.4 Write a docx with citations

If you write in Word, type Pandoc-style placeholders directly:

- `[@SmithCrystal2024]` — one citation
- `[@Smith2024; @Doe2023]` — multiple in one bracket
- `[@Smith2024, p. 42]` — with a page locator

Drag the `.docx` onto the Pohti page. Pohti unzips it (fflate), walks every paragraph in `word/document.xml`, replaces every `[@key]` with a formatted citation in your chosen style, and appends a bibliography section at the end. You get a new file `<name>-cited.docx` downloaded to your Downloads folder. Missing citekeys show as `[?]` in the document and are listed in the post-process toast so you can find them.

This works without installing Pandoc. If you prefer Pandoc, the export menu also has a *Use with Word + Pandoc...* entry that gives you the exact command to copy-paste.

4.5 Recover after switching browsers, ports, or machines

Browser storage is keyed by *origin*: `http://127.0.0.1:8765` and `http://127.0.0.1:8766` are different origins, so they have different IndexedDBs. This is the most common “my library disappeared” scenario.

Pohti's defense: when a library folder is linked, the auto-write of `_pohti-library.json` mirrors the entire library to disk on every change (debounced). On a fresh origin, you re-link the folder; Pohti detects the backup file, asks “Found a library backup with N entries (last saved <timestamp>). Restore?”, and replays it into the new IndexedDB. The plain JSON on disk is the source of truth; the browser cache is rebuilt from it.

5 An honest comparison

The table is a guide, not a marketing piece. Zotero with Better BibTeX is the right tool for most researchers; if it works for you, do not switch. Pohti exists because a few of those columns mattered

	Zotero+BBT	JabRef	Mendeley	EndNote	Pothi
Free / open-source	yes / yes	yes / yes	yes / no	no / no	yes / yes
No account required	yes (BBT)	yes	no	no	yes
Local-only by default	no	yes	no	yes	yes
Plain text on disk	no	yes (.bib)	no	no	yes (JSON + .bib)
Manuscript-first model	no	no	no	no	yes
Per-citation rationale	no	no	no	no	yes
.bib lives next to manuscript	opt-in (BBT)	yes	no	no	yes
Live-sync .bib on edit	on save (BBT)	on save	no	no	yes (4 s)
Watch-folder auto-import	via ZotFile	no	no	no	yes
Drag-PDF DOI sniff	yes	no	yes	no	yes
docx “cite-while-write”	yes (Word add-in)	no	yes	yes	via Pandoc bridge
Custom entry types	via plugin	yes	no	yes	via schema.js
Runs in browser	web only	no	no	no	yes
Install ceremony	app + ext + plugin	app + Java	app + ext	app	none (browser)
Lines of source	~300 k	~200 k	closed	closed	~10 k

Table 1: Pothi compared to widely used reference managers. “Yes” / “no” cells are the author’s reading; please correct any errors.

to me enough to write something different.

6 What it does not do

Honesty matters here.

- **No Microsoft Word add-in.** The docx pipeline works through a drop-and-replace flow, not a live task pane inside Word. Building a real Office add-in is a separate sub-project (different deployment, Office.js, manifest distribution) and is not in this version.
- **Chromium-only for folder linking.** The File System Access API is not yet in Firefox or Safari. The non-folder parts of Pothi — adding references manually, browsing, search, exports — work in any modern browser. Folder linking, watch-folder scan, and PDF attachments need Chrome / Edge / Brave / Opera.
- **No cloud sync.** Use Nextcloud / Dropbox / iCloud / git on the linked folder. The plain-JSON `_pothi-library.json` backup file is the synchronization unit; conflicts surface as file conflicts and you resolve them in your sync tool.
- **No real-time multi-user.** If you open Pothi in two tabs at the same origin they will not see each other’s edits until reload. Two machines editing the same folder concurrently will produce a sync conflict on the backup file.
- **Single maintainer.** I wrote this. If I get hit by a bus, the code is MIT-licensed and small enough to fork.
- **Citation styles are limited.** The docx pipeline ships with two styles, author-year (APA-ish) and numeric (IEEE-ish). Bring-your-own CSL support is a future addition; Pandoc handles it

today.

- **Semantic Scholar** is **rate-limited** without an API key, so the citation-count chip is best-effort.

7 Repository, installation, and license

Code. <https://github.com/<REPO>/pothi> (replace with the real URL).

License. MIT. The file `LICENSE` in the repository carries the full text. Do anything you want with the code; just keep the copyright line.

Install. Clone the repository. From the project folder:

```
./install.sh # Linux / macOS  
install.bat # Windows
```

This drops a `pothi` command into your `PATH` and creates a clickable launcher (`pothi.desktop` on Linux, `Pothi.command` on macOS, `Pothi.bat` on Windows). After running it, reload your shell and type `pothi`; the local server starts and your browser opens.

Manual run. If you would rather not install:

```
python3 -m http.server 8765 --bind 127.0.0.1
```

Then open <http://127.0.0.1:8765/> in a Chromium-based browser.

8 Future and open invitations

Things on my list, in no particular order:

- A real Microsoft Word add-in (Office.js task pane).
- PDF annotation extraction — pull highlights via PDF.js, surface them as quotable snippets attached to entries.
- Bring-your-own CSL — vendor citeproc-js so the docx pipeline handles any of the 10 000+ styles at <https://www.zotero.org/styles>.
- Integration with experiment / structure databases (in my case, crystallography records). Plain-text on disk makes this trivial in principle but needs a target database to demonstrate.
- A polished onboarding tour and a single-file bundled build for double-click usage.

If you find Pothi useful, the most helpful contributions are: bug reports with reproduction steps, screenshots of the UI breaking on your browser, and `.bib` files where the import either dropped data or got the Unicode wrong. Pull requests welcome.

Acknowledgements

This project stands on the shoulders of [Preact](#), [htm](#), [PDF.js](#), [flate](#), [CrossRef](#), [Semantic Scholar](#), [OpenLibrary](#), and [Pandoc](#). The local-first framing is heavily influenced by the Ink & Switch [Local-First Software](#) essay. The manuscript-first idea — that a paper is the natural unit of research curation, not the library — is, as far as I know, original to this project, but I would be surprised if no one else has independently arrived at it.